

---

# MEM-PREM: A MEMORY-INFORMED PRE-EMPTION MODEL FOR SCHEDULING LLM REQUESTS

---

**Anmay Gupta\***

School of Engineering and Applied Sciences  
Harvard University  
Cambridge, MA 02134  
anmaygupta@college.harvard.edu

**Emmanuel Rassou\***

School of Engineering and Applied Sciences  
Harvard University  
Cambridge, MA 02134  
emmanuel\_rassou@college.harvard.edu

**Kitty Wang\***

School of Engineering and Applied Sciences  
Harvard University  
Cambridge, MA 02134  
kittywang@college.harvard.edu

May 10, 2025

## ABSTRACT

This work initiates a new line of research into memory-informed scheduling policies that move beyond static thresholding via hyperparameter tuning. We first introduce dynamic threshold-based approaches and demonstrate that two of our proposed functional forms—parabolic and exponential—consistently match or exceed the performance of prior state-of-the-art methods across a wide range of workload scenarios. Building upon this, we explore smoother integrations of memory-awareness into scheduling decisions through parametric penalty functions. Our implementations using exponentiated and sigmoid penalties rely on simple yet effective memory heuristics. Furthermore, we analytically extend the closed-form expression for mean response time to accommodate both dynamic thresholds and penalty-based policies. These contributions broaden the design space for future scheduling algorithms, enabling principled navigation of the tradeoff between response time and peak memory consumption.

## 1 Introduction

Latency is the currency of user engagement. A *single-digit* percentage reduction in mean response time at the scale of modern chat assistants ( $\mathcal{O}(10^6)$  requests per day) translates into millions of dollars of compute savings and, more importantly, into perceptible fluidity that keeps users in the conversational loop. Yet the very mechanism that accelerates Transformer inference—the key-value (KV) cache—turns each pre-emption into a memory liability: every interrupted request retains a growing slice of embeddings that competes for scarce GPU VRAM. When memory fills, the scheduler must *evict* caches and recompute them later or *swap* them to host RAM, either of which erases the latency gain that motivated the pre-emption in the first place.

Existing schedulers acknowledge this tension only implicitly. TRAIL and LAMPS, for instance, rank requests by their *predicted* remaining work and block pre-emptions after a fixed fraction of progress, commonly  $C = 0.8$  of the original estimate [1]. That static cliff performs well under the particular workload for which it was tuned, but falters whenever memory pressure intensifies or prediction error grows. What is missing is a principled mechanism that scales the aggressiveness of pre-emption with *observed* memory utilization while preserving the analytical transparency that made age-based policies so attractive under SOAP analysis [2].

---

\*Equal Contribution

We introduce *MEM-PREM*, a memory-informed pre-emption framework that couples rank-based scheduling with an explicit penalty for VRAM consumption. Our central idea is to embed the current memory footprint into the rank function itself, thereby allowing a job’s priority to decay *continuously* as its resident cache grows. In the limit of abundant memory MEM-PREM collapses to SPRPT, matching the optimal response-time behaviour of size-aware policies, while under high pressure it converges toward SJF, finishing outstanding requests to free space.

We extend the SOAP framework to accommodate state-dependent penalties and derive a closed-form expression for the mean response time of any MEM-PREM instantiation. Two concrete specializations illustrate the power of the framework: *Dynamic Threshold SPRPT*, which replaces the fixed cut-off  $C_r$  by a smooth, memory-aware knee curve, and *Parametric Penalty SPRPT*, which subtracts a memory-scaled discount from the predicted remaining time. Simulations on both synthetic (Exponential–Exponential, burst) and realistic (Alpaca-derived) workloads show up to a  $\times$  reduction in mean response time relative to the best static baseline while cutting peak memory usage by . Because the penalty function is analytic, MEM-PREM inherits the fault-tolerance guarantees of SOAP-bubble scheduling [2] and retains compatibility with online Bayesian size-prediction modules. Overall, MEM-PREM offers a single, tunable knob that reconciles the historically competing objectives of *fast tokens* and *slim memory*.

| Algorithm                 | Completed Requests | Avg. Wait Time (s) | Mean Resp. Time (s) | Peak Mem. (MB) | Preemptions |
|---------------------------|--------------------|--------------------|---------------------|----------------|-------------|
| FCFS                      | 14792              | 15.318             | 16.887              | 2378           | 0           |
| SJF                       | 14792              | 7.397              | 11.737              | 5028           | 6494        |
| SPRPT                     | 14792              | 7.950              | 10.874              | 2952           | 2835        |
| LSPRPT                    | 14792              | 7.961              | 10.860              | 2942           | 2613        |
| DTPRPT (hyperbola, 0.2)   | 14792              | 11.480             | 17.827              | 11969          | 5548        |
| DTPRPT (parabola, 0.6)    | 14792              | <b>8.377</b>       | 10.552              | <b>2378</b>    | 1440        |
| DTPRPT (exponential, 0.3) | 14792              | 8.408              | 10.517              | <b>2378</b>    | 1119        |
| PSPRPT 1 (exponentiated)  | 14792              | 8.892              | 10.482              | <b>2378</b>    | 155         |
| PSPRPT 2 (sigmoid)        | 14792              | 8.902              | <b>10.472</b>       | <b>2378</b>    | <b>14</b>   |

Table 1: Performance comparison across scheduling algorithms under Poisson workload

## 2 Background

### 2.1 Scheduling Theory

Arrival Rate ( $\lambda$ ), Serving Rate (token generation rate) Standard queueing system assumptions: poisson arrivals of rate  $\lambda < 1$ , independent service and prediction times.

In interactive computing systems, scheduling jobs with unknown job-sizes with an objective to minimize the mean response time is a classic problem. Although a number of scheduling policies requires the knowledge of each incoming job’s exact age at every moment in time in order to be implemented, schedulers present in some computer systems only tracks each job’s approximate age since constantly updating age information, especially in a distributed network context, incurs a significant delay or cost.

### 2.2 Memory growth during LLM inference

During LLM inference, a natural language prompt is provided by the user, for which the LLM needs to autoregressively generate tokens to continue the prompt, with the goal of answering the user’s question or instruction. Self-attention as used in Transformers [3] learns relevance weights between pairs of tokens. Every *Query* vector can be represented as a combination of the *Key* and *Value* vectors of the other tokens. These vectors are computed with a matrix multiplication from the Key, Query and Value weight matrix.

Inference is represented by two phases: Prefilling and Decoding. Prefilling consists of calculating the attention scores between pairs of tokens in the original prompt. The Decoding phase is responsible for generating the next tokens, and so for a sequence of length  $N$ , it requires  $O(N^2)$  computations, however, a lot of these computations are redundant since the previous token’s key and value will have the same weight matrix applied to it. This motivates the *Key-Value Cache* as a performance boost for inference speeds at a cost of memory usage.

In state-of-the-art systems, every key and value vector is henceforth cached to improve response times. Every new token generated requires that its key and value vector be stored in the cache, while the query vector is unique to that

particular decoding step, and as such can be discarded. The memory growth of the decoding phase can be modeled as proportional to the number of tokens generated so far, known as its *age* in the literature. The prefilling stage is more complicated to model in realistic settings, since recent work has shown that this stage can be processed in parallel, and also overlap with the Decoding phase. Added with the natural observation that most responses are significantly longer than the original prompt, we will focus on the more theoretical-friendly decoding phase, and leave the inclusion of prefilling to orthogonal lines of work.

### 2.3 Common Scheduling Policies

**First Come First Serve(FCFS)** is the most naive way to schedule incoming requests. For high workloads, this policy has one major flaw, termed as "head-of-line blocking". This is when a very large request arrives before several short requests, and ends up making them wait. Intuitively, if we have request  $A$  has time  $t_A$  and request  $B$  has time  $t_B$  with  $t_A \gg t_B$ , then serving  $A$  before  $B$  results in a mean response time of  $\frac{t_A + (t_A + t_B)}{2}$ , while prioritising the shorter request results gives a more optimal  $\frac{t_B + (t_A + t_B)}{2}$ . This policy only becomes optimal when job sizes are completely unknown.

**Shortest Job First(SJF)** attempts to fix the shortfall of FCFS by serving the shortest job first. Under burst workloads where all request lengths are observed before making a decision, this performs near optimal. Under natural settings such as a Poisson Process, the requests arrive over time, and so a job request may be the shortest at the time of its scheduling, but a new shorter job can arrive during its processing, which recreates the same head-of-line blocking experienced by FCFS.

**Shortest Remaining Processing Time(SRPT)** outperforms SJF when requests arrive over time. SRPT considers the remaining processing time of the active requests as well as waiting requests, and can *preempt* the currently served job if a shorter one arrives. As such, it is also known as preemptive shortest job first or PSJF. A pre-empted goes back to into the waiting queue, however, its progress is not reset. To calculate the remaining processing time, one needs to keep track of the *age* of each request, which starts out at 0, but increases as a function of the number of tokens that have been served.

**Shortest Predicted Remaining Processing Time (SPRPT)** is the same policy as SRPT but adapted to cases when one does not have perfect request lengths, and as such must predict them, susceptible to errors. There can be adversarial scenarios that arise when a predicted short job could cause a head-of-line blockage. Optimal performance under these scenarios measures robustness and fault tolerance. When peak memory usage is an orthogonal objective, the act of preempting a request to reduce mean response time is not a free lunch anymore. In the LLM scheduling setting, these preempted requests retain their key-value cache, and accumulate overtime. To prevent out-of-memory issues in real workloads, it is equally important to also finish completing requests in order to free memory to be used by the next served request.

**Limited Shortest Predicted Remaining Processing Time (LSPRPT)** by Shahout et al. [1] observes the heuristic that mature requests predicted to be almost completed, should be immune to preemption. Otherwise the key-value cache already computed will have to be stored until the jobs with higher priority complete. The threshold is given in terms of  $t_{limit} \times \text{age}$ , where  $t_{limit}$  is a hyperparameter that measures how strict the threshold. A threshold of  $t_{limit} = 0.8$  is shown to be a good choice, which means during the last 20% of a requests serving, it cannot be interrupted.

One interpretation of this policy is as an interpolation between SJF and SPRPT. As the preemption threshold is lowered, the number of preemptions decrease, until one serves the shortest job until completion at the extreme. As the preemption threshold is increased, the number of preemptions increase as well, until there is no limiting of preemptions when  $t_{limit} = 1.0$ . Note that this analysis holds true for perfect predictions, but becomes more complicated when prediction errors are introduced. The static nature of such a threshold opens up opportunity to dynamically tune it in terms of the active memory usage.

### 2.4 Metrics

**Mean Response Time** represents the mean latency from when each request arrives till its completion. An alternative metric in the literature is Median Response Time, however, this does not take into account outliers where some requests may have been treated unfairly. Mean Response Time, where outlier times also need to be reduced, fits the setting of LLM scheduling, given that fairness and consistency need to be upheld when completing requests from different users.

**Average Waiting Time** is a related metric, but only measures the time till the request is first served. Given that the prefilling stage is not incorporated, this metric becomes equivalent to Time To First Token (TTFT). While this separates

the length of the actual request with how effect the scheduling is, it also does not incorporate future preemptions of the request in the waiting time, which limits its informativeness for the scheduling policies researched in this paper.

**Peak Memory** measures the maximum memory at any given time. It is equally important to reduce this peak since Out-Of-Memory(OOM) errors during LLM scheduling are much more critical than a slightly slower response time. While memory is measured in bytes in practice, the main factor contributing to memory consumption during inference is the growth of the Key-Value cache, especially for requests with long responses. The peak memory can thus alternatively be measured in tokens generated across active and preempted requests. Note that once a request completes, the memory can immediately be freed, and so the memory usage decreases.

The challenge in the space results from the dual-objective of reducing Response and Waiting Time as well as Peak Memory. Each policy represents a unique point on this pareto frontier, with the most useful ones navigating the tradeoff space in a more optimal way.

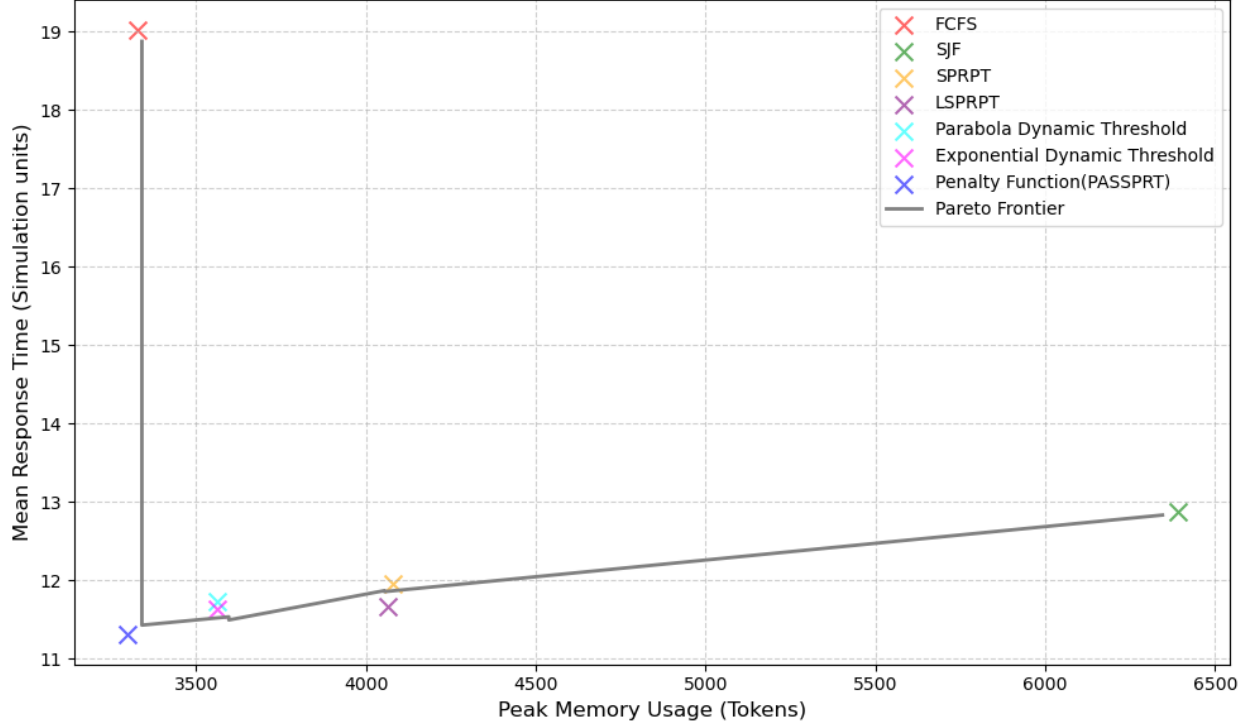


Figure 1: Example of a scheduling policy pareto frontier for a specific synthetic workload. The Dynamic Threshold policies are our contributions.

## 2.5 SOAP Analysis

Generalizing the Gittins policy paradigm, Scully et al. unified such policies in the SOAP (Schedule **O**rded by **A**ge-based **P**riority) family, a category of policies for an M/G/1 queue in which a job’s priority, or its *rank*, is dependent on its age according to a *rank function*. Formally, a rank function

$$r : D \times \mathbb{R}_{\geq 0} \rightarrow R, \quad r(d, a) = \langle r_1(d, a), \dots, r_m(d, a) \rangle, m \in \mathbb{N},$$

maps each job state to a vector that is compared lexicographically. At every instant, the server works on the job of *minimal* rank [2]. Classic policies such as FCFS, SRPT, FB, class priorities, as well as intricate schemes like the Gittins-index rule and “checkpoint-preemptible” variants, can all be encoded by a suitable  $r$  under this unifying framework of SOAP.

While previous analytic techniques handled only *monotone* rank functions, many realistic policies employ rank functions that are *non-monotone* in age, meaning the usual “tagged-job” busy-period arguments break down. In particular, two concerns emerge:

1. A tagged job’s rank can *increase* later, so arrivals that are initially irrelevant may pre-empt it subsequently.
2. Old jobs’ ranks can oscillate above and below the tagged job’s worst future rank, making their residual interference hard to bound.

Because priority depends on present age, performance can be analyzed by tracking a tagged job  $J$  through the system. Assuming Poisson arrivals with rate  $\lambda$ , i.i.d. job sizes  $X$ , and a system load  $\rho = \lambda \mathbb{E}[X]$ , for a tagged job  $J$ , we define its *worst future rank* as

$$r^*(a) = \sup_{b \in [a, x_J)} r(d_J, b),$$

the highest rank  $J$  will ever attain after age  $a$ . Job  $J$ ’s delay due to *any* other job ends as soon as that job either completes or surpasses  $r^*$ , enabling us to treat  $J$  as if it always had rank  $r^*$ . This is the pessimism principle at the core of SOAP analysis.

Old jobs are partitioned with respect to  $r^*$  into classes of *discarded*, *original*, where a job has rank  $\leq r^*$  from birth, and *recycled*, where a job’s rank dips below  $r^*$  after previously exceeding it. As a result of this construction,  $J$ ’s delay from old work equals the FCFS queueing time in an M/G/1 system with *sparse vacations*. Replacing  $J$ ’s true rank by  $r^*$  freezes its priority, allowing the classical split

$$T_{d,x} = T_{d,x}^{\text{wait}} + T_{d,x}^{\text{res}},$$

but with *new* effective job sizes. The Laplace-Stieltjes transforms then give closed forms for both components, culminating in the transform of total response time. Integrating those transforms yields a remarkably compact universal expression. Let

$$\begin{aligned} c[r] &= \inf\{a \geq 0 : r(a) \geq r\} \\ X^{\text{new}}[r] &= \min\{X, c[r]\}, \\ (b_0[r], c_0[r]) &= (0, c[r]), \\ (b_i[r], c_i[r]) &= (\inf\{a > c_{i-1}[r] : r(a) \leq r\}, \inf\{a > b_i[r] : r(a) > r\}) \quad (i \geq 1), \\ X_i^{\text{old}}[r] &= \begin{cases} 0, & X \leq b_i[r], \\ X - b_i[r], & X \in (b_i[r], c_i[r]), \\ c_i[r] - b_i[r], & X \geq c_i[r]. \end{cases} \end{aligned}$$

We derive that for any SOAP policy, a job with descriptor  $d$  and size  $x$  has mean response

$$\mathbb{E}[T_{d,x}] = \frac{\lambda \sum_{i \geq 0} \mathbb{E}[X_i^{\text{old}}[r]^2]}{2(1 - \rho_0^{\text{old}}[r])(1 - \rho^{\text{new}}[r])} + \int_0^x \frac{da}{1 - \rho^{\text{new}}[r^*(a)]},$$

where

- $r = r^*(0)$  and  $r^*(a) = r_{d,x}^{\text{worst}}(a)$ ,
- $\rho^{\text{new}}[r] = \lambda \mathbb{E}[X^{\text{new}}[r]]$ ,  $\rho_i^{\text{old}}[r] = \lambda \mathbb{E}[X_i^{\text{old}}[r]]$ ,
- $X^{\text{new}}[r]$  is the service a *new* arrival contributes until it either finishes or exceeds rank  $r$ ,
- $X_i^{\text{old}}[r]$  is the service an *old* job contributes during its  $i$ -th “original/recycled” interval below rank  $r$ .

The first term is the Pollaczek–Khinchine factor for waiting, augmented by the vacation effect from recycled work to quantify delay caused by *old* jobs that were already present when  $J$  arrived, and the integral captures the accumulation of residence time as  $J$ ’s worst future. Each competing job delays  $J$  only until it either completes or its rank exceeds  $r^*(a)$ , in which case it exits the *new- $r$*  interval  $[0, c[r])$  or, for jobs in progress, the successive  *$i$ -old* intervals  $(b_i[r], c_i[r])$ .

### 3 Method

#### 3.1 Motivation from Previous Work

Although preemption can be implemented at inference time in LLM systems to shorten response times, each interruption increases memory usage and can backfire when the system nears its memory limit. In LLM serving, a full memory

pool forces us either to evict jobs, forcing the system to recompute them later, or to migrate key–value (KV) cache blocks from GPU to CPU. Eviction results in redundant computation, while swapping stalls the model’s forward-pass and delays the entire batch. Hence, preemptions should be curtailed once a job is heavily resident in memory: early interruptions are relatively cheap, but late-stage ones are costly.

TRAIL enforces this idea with a simple rule. Given a job’s initial runtime estimate,  $r$ , taken as the midpoint of its predicted size bin, the scheduler permits preemption only during the first  $\lfloor Cr \rfloor$  decoding iterations, where  $C$  is a fixed constant. As the job approaches completion and its memory footprint grows, further preemptions are inhibited, preventing memory pressure from undermining overall performance. Recognizing similar dependency limitations observed in works such as *Perception in Advance* [4] and the resource consumption rate limiting the efficiency and practicality of Zheng et al.[4], TRAIL (Token Response and Embedding Adaptive Inference Layer) introduces an improved approach that reduces the response time in LLM inference through a combination of two integral contributions:

1. highly accurate, iteration-level predictions request lengths that requires low overhead to allow for size-based scheduling, and
2. preemption scheduling at the token granularity, accounting for memory overhead, a heuristic not considered in previous analyses.

However, TRAIL’s method of determining the threshold to completely disable preemption is by an absolute cliff based on empirical results. In particular, the current threshold for preemption disabling is simply  $a_0$ , defined as

$$a_0 = C \cdot r,$$

where  $r$  is the predicted request length and  $C$  is a tunable, but fixed constant. From empirical experimentation [1], Shahout et al. have concluded that a value of  $C = 0.8$  is optimal under select setup and implementation details. Extending this principle of a preemption threshold, we ask the question of generalization. We explore the application of SOAP analysis to two approaches to a more generalized thresholding mechanism — dynamic thresholding and parametric penalty — in the formalization of our methods. As a preview of coming attractions, we investigate the mean response time of an incoming job  $J$  with descriptor  $d$  and size  $x$ , which, under the TRAIL mechanism, is given by

$$\mathbb{E}[T(x, d)] = \frac{\lambda \cdot \sum_{i=0}^{\infty} \mathbb{E}[X_i^{old}[r_{worst}]^2]}{2(1 - \lambda \mathbb{E}[X_0^{old}[r_{worst}]]) (1 - \lambda \mathbb{E}[X^{new}[r_{worst}]])} + \int_0^x \frac{1}{1 - \lambda \mathbb{E}[X^{new}[rank_{d,J}^{worst}(a)]]} da$$

The extended decomposition and proof of this statement is presented in the Appendix C of TRAIL (Shahout et al., 2024)[1, 2].

### 3.2 Dynamic Threshold

As our first presented approach to generalize the preemption thresholding mechanism, we introduce a parametric threshold function, replacing  $a_0$ .

We begin by describing the rank function for a job with descriptor  $(x, r, a)$ , where  $x$  is the actual job size,  $r$  is the approximate predicted size, and  $a$  is the job’s age. We define the rank function

$$\text{rank}(x, r, a) = \begin{cases} r - a & \text{if } a < h(a, r, m) \\ -\infty & \text{otherwise} \end{cases}$$

where  $h$  in this case serves as the penalty function that accepts parameters  $a, r$ , as well as  $m$ , the overall memory capacity of the system. Because the rank function is monotonic, a job’s future rank is simply its initial prediction:

$$\text{rank}_{worst}(a) = \begin{cases} r - a & \text{if } a < h(a, r, m) \\ -\infty & \text{otherwise.} \end{cases}$$

When  $a = 0$ , the rank function can easily be expressed as  $r_{worst} = \text{rank}_{worst}(0) = r$ . This allows us to express the expected mean response time under this dynamic threshold model

**Theorem 1** *For SPRPT with a parametric penalty function as the preemption limiting mechanism, where above age  $h(a, r, m)$  the jobs become non-preemptable, the expected mean response time for a job of true size  $x$  and predicted*

size  $r$  can be expressed as

$$\mathbb{E}[T(x, r)] = \frac{\lambda \cdot \left( \int_{r_I=r+h(a,r,m)}^{\infty} \int_{x_I=r_I-r}^{\infty} g(x_I, r_I) \cdot (x_I - (r_I - r))^2 dx_I dr_I \right)}{2(1 - \rho'_r)^2} + \int_0^x \frac{1}{1 - \rho'_{(r-a)^+}} da.$$

where we define

$$\rho'_r = \lambda \int_0^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy.$$

In our analysis, we calculate  $X^{\text{new}}[\text{rank}_{\text{worst}}(x, r, a)]$ ,  $X^{\text{old}}[\text{rank}_{\text{worst}}]$ , and  $X_i^{\text{old}}[\text{rank}_{\text{worst}}]$ . To begin proving that the expression for the expected mean response time holds true for our generalized parametric penalty framework, we begin with computation of  $X^{\text{new}}[\text{rank}_{\text{worst}}(x, r, a)] = X^{\text{new}}[r - a]$ :

Suppose that a new job  $K$  of predicted size  $r_K$  arrives when  $J$  has age  $a$ . If  $K$  has a predicted job size less than the remaining processing time or rank for  $J$  in the case of  $r_K < r - a$  and  $a < h(a, r, m)$ , then  $K$  will outrank  $J$ . Namely, this yields

$$X_{x_K}^{\text{new}}[r - a] = x_K \mathbb{I}(r_K < r - a) \mathbb{I}(a < h(a, r, m))$$

$$\mathbb{E}[X^{\text{new}}[r - a]] = \int_0^{r-a} \int_{x_K=0}^{\infty} x_K \cdot g(x_K, y) dx_K dy.$$

From here, we compute  $X_0^{\text{old}}[r]$ . Let  $I$  be some old job that is currently processing in the system. Whether job  $I$  is then considered an original or recycled job depends on its predicted size relative to  $J$ 's predicted size. If it is the case that  $r_I \leq r$ , then job  $I$  is original until its completion as its rank will never exceed  $r$  to induce a flip, and in the alternative case where  $r_I > r$ , then job  $I$  is recycled. This allows us to express  $X_{x_I}^{\text{old}}[r]$  and its expectation as follows:

$$X_{x_I}^{\text{old}}[r] = x_I \mathbb{I}(r_I \leq r)$$

$$\mathbb{E}[X^{\text{old}}[r]] = \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy$$

$$\mathbb{E}[(X_0^{\text{old}}[r])^2] = \int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy$$

Now that we have computed  $E[(X_0^{\text{old}}[r])^2]$ , we move on to computing  $X_i^{\text{old}}[r_{\text{worst}}]$ . Recall that given an old job  $I$  present in the system, job  $I$  becomes recycled when

$$r_I - a = r$$

is achieved. Rearranging this equation, we obtain that at age  $a = r_I - r$ , the job will be served until completion only if  $a < h(a, r, m)$ , which will be  $x_I - a_I = x_I - (r_I - r)$ . As a result, we have that  $X_1^{\text{old}}[r_{\text{worst}}] = x_I - a_I = x_I - (r_I - r)$ . Taking the expectations, we have

$$\mathbb{E}[(X_1^{\text{old}}[r])^2] = \int_{r_I=r+h(a,r,m)}^{\infty} \int_{x_I=r_I-r}^{\infty} g(x_I, r_I) \cdot (x_I - (r_I - r))^2 dx_I dr_I$$

Applying Theorem 5.5 of SOAP [5] yields a final mean response time of jobs with descriptor  $(x, r, a)$  as given. Letting

$$\rho'_r = \lambda \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy,$$

we can write

$$\mathbb{E}[T(x, r)] = \frac{\lambda \cdot \left( \int_{r_I=r+h(a,r,m)}^{\infty} \int_{x_I=r_I-r}^{\infty} g(x_I, r_I) \cdot (x_I - (r_I - r))^2 dx_I dr_I \right)}{2(1 - \rho'_r)^2} + \int_0^x \frac{1}{1 - \rho'_{(r-a)^+}} da.$$

### 3.3 Parametric Penalty Function

Taking a different perspective, we also explore smoothing out the threshold curve by implementing a parametric penalty function.

We begin by describing the rank function for a job with descriptor  $(x, r, a)$ , where  $x$  is the actual job size,  $r$  is the approximate predicted size, and  $a$  is the job's age. Instead of defining a piecewise mapping for the rank function, we define

$$\text{rank}(x, r, a) = r - h(a, r, m),$$

where  $h$  in this case serves as the penalty function that accepts parameters  $a, r$ , as well as  $m$ , the overall memory capacity of the system.

**Theorem 2** *For SPRPT with a parametric penalty function as the preemption limiting mechanism, where above age  $r - h(a, r, m)$  the jobs become non-preemptable, the expected mean response time for a job of true size  $x$  and predicted size  $r$  can be expressed as*

$$\begin{aligned} \mathbb{E}[T(x, r)] = & \frac{\lambda \cdot \left( \int_{r_I=r}^{\infty} \int_{x_I=h^{-1}(r_I-r, r, m)}^{\infty} g(x_I, r_I) \cdot (x_I - h^{-1}(r_I - r, r, m))^2 dx_I dr_I \right)}{2 \left( 1 - \lambda \left( \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy \right) \right) \left( 1 - \lambda \left( \int_0^{r-h(a, r, m)} \int_0^{\infty} x_K \cdot g(x_K, y) dx_K dy \right) \right)} \\ & + \int_0^x \frac{1}{1 - \rho'_{(r-a)^+}} da, \end{aligned}$$

where we define

$$\rho' = \lambda \int_0^{r-h(a, r, m)} \int_0^{\infty} x_I \cdot g(x_I, y) dx_I dy.$$

In our analysis, we calculate  $X_{\text{rank}_{\text{worst}}(x, r, a)}^{\text{new}}$ ,  $X_{\text{rank}_{\text{worst}}}^{\text{old}}$ , and  $X_i^{\text{old}}[\text{rank}_{\text{worst}}]$ . To begin proving that the expression for the expected mean response time holds true for our generalized parametric penalty framework, we begin with computation of  $X_{\text{rank}_{\text{worst}}(x, r, a)}^{\text{new}} = X^{\text{new}}[r - h(a, r, m)]$ :

Suppose that a new job  $K$  of predicted size  $r_K$  arrives when  $J$  has age  $a$ . If  $K$  has a predicted job size less than the penalty-adjusted remaining processing time or rank for  $J$  in the case of  $r_K < r - h(a, r, m)$ , then  $K$  will outrank  $J$ . Namely, this yields

$$\begin{aligned} X_{x_K}^{\text{new}}[r - h(a, r, m)] &= x_K \mathbb{I}(r_K < r - h(a, r, m)) \\ \mathbb{E}[X^{\text{new}}[r - h(a, r, m)]] &= \int_0^{r-h(a, r, m)} \int_{x_K=0}^{\infty} x_K \cdot g(x_K, y) dx_K dy \end{aligned}$$

From here, we compute  $X_0^{\text{old}}[r]$ . Let  $I$  be some old job that is currently processing in the system. Whether job  $I$  is then considered an original or recycled job depends on its predicted size relative to  $J$ 's predicted size. If it is the case that  $r_I \leq r$ , then job  $I$  is original until its completion as its rank will never exceed  $r$  to induce a flip, and in the alternative case where  $r_I > r$ , then job  $I$  is recycled. This allows us to express  $X_{x_I}^{\text{old}}[r]$  and its expectation as follows:

$$\begin{aligned} X_{x_I}^{\text{old}}[r] &= x_I \mathbb{I}(r_I \leq r) \\ \mathbb{E}[X^{\text{old}}[r]] &= \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy \\ \mathbb{E}[(X_0^{\text{old}}[r])^2] &= \int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy \end{aligned}$$

Now that we have computed  $E[(X_0^{\text{old}}[r])^2]$ , we move on to computing  $X_i^{\text{old}}[r_{\text{worst}}]$ . Recall that given an old job  $I$  present in the system, job  $I$  becomes recycled once the equality

$$r_I - h(a_I, r_I, m) = r$$

is achieved and surpassed. Rearranging this equation, we obtain

$$h(a, r, m) = r_I - r \Rightarrow a_I = h^{-1}(r_I - r, r, m)$$

As a result, when substituted in, we have that  $X_1^{\text{old}}[r_{\text{worst}}] = x_I - a_I = x_I - (h^{-1}(r_I - r, r, m))$ . Taking the expectations, we have

$$\mathbb{E}[(X_1^{\text{old}}[r])^2] = \int_{r_I=r}^{\infty} \int_{x_I=h^{-1}(r_I-r, r, m)}^{\infty} g(x_I, r_I) \cdot (x_I - h^{-1}(r_I - r, r, m))^2 dx_I dr_I$$

Applying Theorem 5.5 of SOAP [5] yields a final mean response time of jobs with descriptor  $(x, r, a)$  as given. Letting

$$\rho'_r = \lambda \int_0^r \int_0^\infty x_I \cdot g(x_I, y) dx_I dy,$$

we can write

$$\begin{aligned} \mathbb{E}[T(x, r)] = & \frac{\lambda \cdot \left( \int_{r_I=r}^{\infty} \int_{x_I=h^{-1}(r_I-r, r, m)}^{\infty} g(x_I, r_I) \cdot (x_I - h^{-1}(r_I - r, r, m))^2 dx_I dr_I \right)}{2 \left( 1 - \lambda \left( \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy \right) \right) \left( 1 - \lambda \left( \int_0^{r-h(a, r, m)} \int_0^\infty x_K \cdot g(x_K, y) dx_K dy \right) \right)} \\ & + \int_0^x \frac{1}{1 - \rho'_{(r-a)^+}} da. \end{aligned}$$

## 4 Memory-Informed Policy Models

### 4.1 Dynamic Threshold Modeling

We propose Dynamic Threshold Preemption Remaining Processing Time (DTPRPT), which adjusts the pre-emption threshold dynamically based on memory usage. Let  $m_0$  be the proportion of memory already used by the current and pre-empted requests. Intuitively, when  $m_0$  is small, our pre-emption threshold can be large to allow pre-emptions to speed up response times but when  $m_0$  is larger, the pre-emption threshold should be more strict to force requests to finish and the key-value cache to be freed. We build upon LSPRPT [1] by defining our threshold as  $t_{\text{limit}} \times r$ , with  $r$  being the predicted request length. In that paper,  $t_{\text{limit}} = 0.8$  regardless of  $m_0$ .

**SOAP Compliant Variant:** SOAP requires that the ranking of requests are only dependent on its "type" and its age [5]. The definition of  $m_0$  represent the proportion of total memory usage breaks this constraint, since this requires knowing global memory of the ages of other jobs that have been preempted. To ensure our theory and experimental section are consistent we also propose a soap compliant variant where  $m_0$  is just defined as the ratio of the current job's age to the memory capacity, which is just a constant defined a priori.

$t_{\text{limit}}$  is then a function of  $m_0$ . We use a *threshold plot* to describe the relationship between  $m_0$  vs  $t_{\text{limit}}$  between 0 and 1 on both dimensions. While there may too many degrees of freedom to find the most optimal design, we use inductive biases to build a model for the threshold that is conscious of the growth of memory [6]:

1. When  $m_0 \approx 0$  then  $t_{\text{limit}} = 1$ , i.e. when memory is empty, then there should be preemption of limitation, which represents SPRPT in this case.
2. When  $m_0 \approx 1$ , then  $t_{\text{limit}} = 0$ , i.e. , when memory is almost full, then no preemption should take place in order to free memory as soon as possible, which represents SJF in this case. This bias may be suboptimal in select cases when there are waiting prompts that are almost completed, however, in a setting when  $m_0$  is gradually incremented, such prompts will not likely arise in practice.
3. The function  $t_{\text{limit}}$  should gradually decrease as  $m_0$  increases, however, at some critical point when memory is almost full, the preemption threshold should rapidly decrease. The shape is synonymous with a knee curve, but the critical point and the rate of descent still remain degrees of freedom.

Note the interpolation between SJF and SPRPT depending on the memory  $m_0$ . Based on the shape described above, we hypothesize three possible classes of functions *a priori* to be used in experiments.

#### 4.1.1 Parabolic Dynamic Threshold

We define  $t_{\text{limit}} = c_1 m_0^2 + c_2 m_0 + 1$  where  $c_1, c_2 \in \mathbb{R}$ . This parabola should ideally go through the points  $(1, 0)$ ,  $(0, 1)$  and we define the hyperparameter  $h$  to be a control knob for the point  $(h, 0.8)$  as a point of reference with respect to

LSPRPT.

$$\begin{aligned} c_1 &= \frac{h - 0.2}{h^2 - h} \\ c_2 &= -1 - c_1 \end{aligned}$$

#### 4.1.2 Hyperbolic Dynamic Threshold

Similarly to the above the function  $y = 1/x$  can be transformed to satisfy the inductive biases with the transformation  $y = 1/(x - 1) + 1$ .

$$\begin{aligned} t_{limit} &= \frac{c}{m_0 - 1} + 1 \\ c &= 0.2 - 0.2h \end{aligned}$$

#### 4.1.3 Exponential Dynamic Threshold

We define our hyperparameter  $h$  as merely a scaling factor in the exponent .

$$t_{limit} = \frac{1 - e^{-h+hm_0}}{1 - e^{-h}}$$

**Hyperparameter Caveat:** While it appears that we are just replacing one hyperparameter  $t_{limit}$  with another  $h$ , we will see that most values of  $h$  can outperform the static threshold. In other words, the informativeness of introducing memory can still give benefits even in a badly tuned threshold function.

### 4.2 Parametric Penalty Function Modeling

Instead of modifying  $t_{limit}$  to force preemption immediately once a certain age is hit, we can adopt a smoother penalty function which encourages preemption dependent on memory usage. This leads us to propose Parametric Age-Scaled Shortest Remaining Processing Time (PASSRPT), which computes a parametric function dependent on the age, predicted size, and system’s memory capacity to determine a job’s rank. Using this approach, we can not only control when jobs are preempted but also which jobs run first, minimizing future preemptions as well.

Similar to the dynamic threshold, where the search space is too huge to fully exhaust, we rely on the inductive biases and constraints such that when  $m_0$  approaches 1, the penalty should be disproportionately higher. SPRPT initially considers a rank function of  $y - a$  where  $y$  is the predicted request length.

#### 1. PASSRPT 1: Exponentiated

We propose an implementation of PASSRPT that interpolates between  $y - a$  and  $y - a^2$  depending on the memory available given by  $m_0$ .

We thus define the rank function of our PASSRPT to be:

$$Rank(y, a) = y - a^{1+m_0}$$

#### 2. PASSRPT 2: Sigmoid

Instead of increasing the age as in PASSRPT 1, we consider decaying it for smaller values of  $m_0$ . To make such a penalty function, we thus use a sigmoid function:

$$Rank(y, a) = y - a \frac{e^{m_0 a}}{1 + e^{m_0 a}}$$

## 5 Evaluation

### 5.1 Setup

We use Simpy to simulate our policies for *sim-time* units of time. The serving rate is given by the token generation rate, which we call  $t_g$ . This represents the number of tokens to be generated per unit time. The requests are generated from some underlying distribution, and follow a poisson process for the interarrival times, with  $\lambda$  requests arriving per unit time on average. We denote  $\lambda$  as the *arrival-rate*. We avoid the use of a real-world request length predictor, as we are mainly concerned with comparing the performance of individual policies on a fixed workload. The extra control in stimulating the predictions and its errors gives us opportunities for even more insights.

We set the memory capacity as  $m_0 = 512$  tokens. This affects our memory-informed policies, and for real systems, should be adjusted based on hardware constraints. We tune our policies based on this initial decision, which is in and of itself scaled to an appropriate size based on real-world request lengths such as those observed in Figure 2.

## 5.2 Workloads

We use four different workloads for our evaluation, which take the form of a poisson process with arrival rate  $\lambda$ .

### 1. Exponential-Exponential

Popularized by Mitzenmacher [7], the request lengths are exponentially distributed with unit time 1. The number of tokens should have mean  $t_g$  in that case. The prediction length  $y$  is then exponentially distribution with mean  $x$ , where  $x$  is the true request length in tokens. This gives a nice formulation for the joint distribution of job as  $g(x, y) = \frac{e^{-x-y/x}}{x}$  which can be used for further theoretical analysis of the mean response time.

$$x \sim \text{Expo}(1/t_g)$$

$$y \sim \text{Expo}(1/x)$$

For simulation purposes, we set  $\text{sim-time}=25000$  and ablate  $\text{token-gen-rate} \in \{128, 256, 512\}$  and  $\lambda \in \{0.4, 0.6, 0.8\}$ .

### 2. Exponential-Perfect

This is similar to the above, except that the predictions are perfect. We use this as a baseline to measure fault tolerance of our new policies, however, in real-world inference systems, having perfect predictions are unrealistic.

$$x \sim \text{Expo}(1/t_g)$$

$$y = x$$

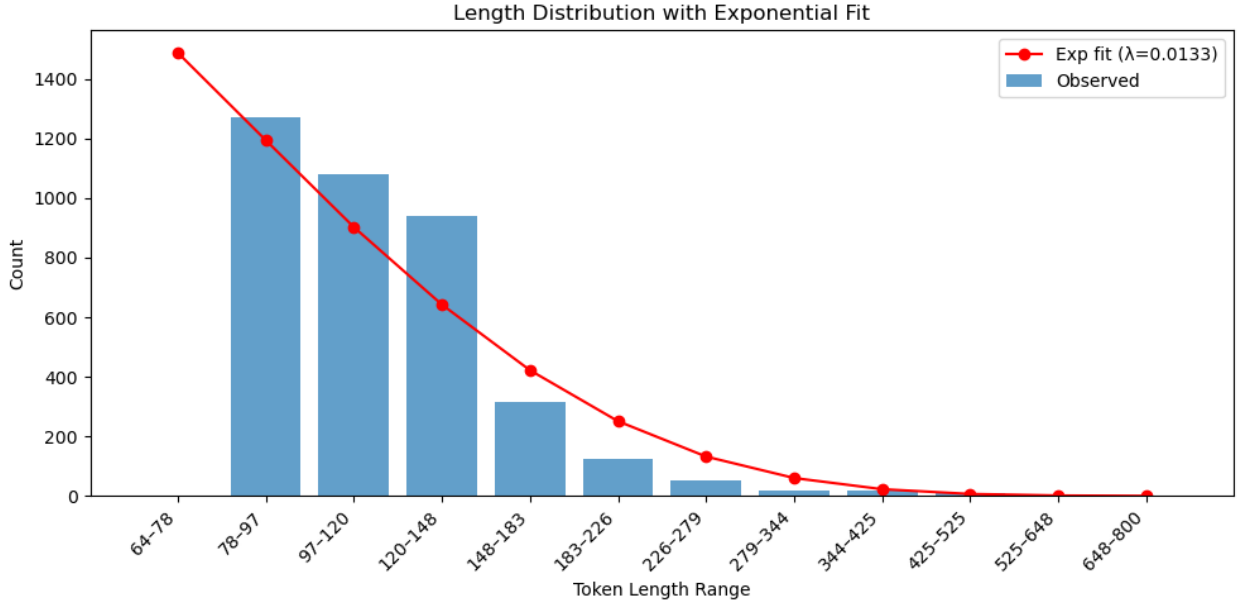


Figure 2: Realistic request length distribution from Llama inference on Alpaca dataset. Denoted as  $H$  for the Realistic-Normal distribution.

### 3. Realistic-Normal

We do inference on Llama-3 8B Instruct[8] using 3,800 queries from the Alpaca Cleaned dataset on Hugging Face [9]. Our request generator then uses the generated lengths as a histogram  $H$ . As shown in Figure 2, the naturally occurring distribution has the shape of an exponential distribution, serving as empirical evidence for our synthetic proxies.

We introduce error with gaussian noise proportional to the request length.  $\sigma$  is then a knob used to control how inaccurate the predictions of our system are.

$$\begin{aligned} x &\sim H \\ y &\sim x + xZ \\ Z &\sim N(0, \sigma^2) \end{aligned}$$

For simulation purposes, we set  $sim-time = 250,000$  to ensure sufficient time for deviation of policies, and we set  $t_g = 256$ ,  $\lambda = 0.6$  and ablate  $\sigma \in \{0.3, 0.5, 0.7\}$

#### 4. Burst workload

The above workloads use a gradual arrival of job requests. To be consistent with Shahout et al. [1], we also consider a sudden burst of requests at the start of a serving period. Using the same parameters for our simulation engine, we set  $sim-time = 100$ ,  $\lambda = 100$ ,  $t_g = 256$  and  $\lambda = 0.6$ . This gives on average 10,000 requests, however, they arrive within a much shorter time period.

### 5.3 DTRPT and PSRPT performance

We test our new policy under rigorous experimentation on all four workloads. The threshold functions can be visualized with the hyperparameters in Figure 5.3.

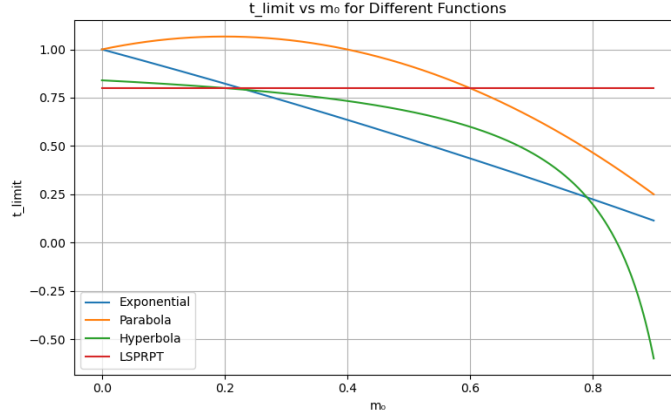


Figure 3: Visualization of DTRPT threshold functions as well as LSPRPT

#### Exponential-Exponential Workload

We graph the results only for  $\lambda = 0.4$  for convenience sake in Table 2, since the differences are small enough to all fit in one graph. Our parabolic and exponential methods perform significantly better in terms of mean response time, and surprisingly under the soap compliant policy, the hyperbolic version joins them.

Since there is significant noise for token lengths of average 128, 256 and 512, it makes sense that LSPRPT beats SPRPT on mean response time. Preempting based on information that is imperfect can slow down the system, while on the contrary limiting preemption policies becomes a form of *fault tolerance*.

The trends are similar for  $\lambda = 0.6$  and  $\lambda = 0.8$ , however, the gains are not as pronounced, especially for  $\lambda = 0.8$ , where there is virtually a tie. As the arrival rate of jobs increase, there becomes a larger selection of jobs to choose from and, thus, the completion order of jobs without any preemption becomes closer to the ascending order of job sizes. This intuition resonates with the results under the Burst Workload.

The waiting time tells a different story. All three of the DTPRPT policies and both of the PSRPT policies perform worse than the SPRPT and LSPRPT. Given that the total response time is faster, but the time till the first token is slower, this shows that our policies have most of their benefits during the processing of a request and not before. This highlights the relative gains of preemption policies such as SPRPT and LSPRPT that DTPRPT and PSRPT try to improve upon.

A lower mean response time is ordinarily attained by storing more tokens in the Key-Value cache, and thus increasing the peak memory usage. Because our DTPRPT policies are memory-informed, they in fact simultaneously achieve a lower peak memory as shown in the third row of Table 2, on top of a lower mean response time. This achieve the same

peak memory as FCFS which has no preemptions, thus proving its optimality for this particular experiment. Combined with the fact that our policies have less total preemptions than LSPRPT and SPRPT, this highlights the principle that preempting carelessly does not always lead to better performance. It is in fact dependent on the specific split in time that dictates whether it is more optimal for response time and peak memory to preempt or not. This is especially evident with the PSPRPT policies, which perform extremely few preemptions but are able to achieve the same mean response time and peak memory usage at the expense of a longer average wait time.

Our most unstable policy is our hyperbolic, which has asymptotes at the two extremes of memory. This instability shows itself even at the  $\lambda = 0.4$  level, however, the SOAP compliant version improves the model despite its proxy for memory usage being more limited. We attribute this surprising result to the fact that a change in one job's age will affect the rank of all waiting jobs in the non-SOAP compliant case, which creates this instability. While SOAP is mostly a theoretical tool, its principle of simplicity and independence in scheduling theory shows its value when designing such policies in practice.

### Exponential-Perfect Workload

Under this workload, our DTRPT policies achieve marginally slower mean response time, however, we still have the same peak memory for the exponential and parabola policies as shown in 5.3. This workload shows that our policies shine when there is a lack of information from the request length due to their reliance on information from memory usage to beat the state-of-the-art policies under exponential-exponential workload.

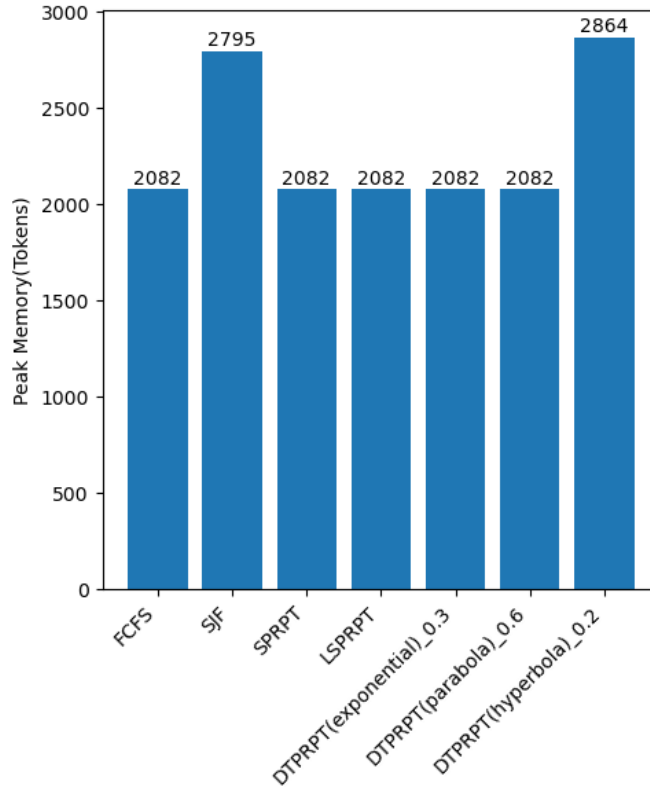


Figure 4: Peak Memory usage for exponential-perfect workload with  $\lambda = 0.6$

### Realistic-Normal Workload

In workloads characterized by heavy-tailed request length distributions, scheduling decisions tend to have a relatively minor effect on response times. This contrasts with exponential distributions, where decisions such as preempting one request in favor of another with significantly different length can have a more pronounced impact. As illustrated in Figure 5.3, all three of our DTRPT policies achieve optimal peak memory usage, matching that of the FCFS policy, which performs no preemptions. However, for certain workload configurations, this memory efficiency comes at the cost of a slight increase in response time. This illustrates a fundamental tradeoff between memory optimization and

performance speed under realistic workload conditions. PSPRPT achieves similar results, as seen in tables 3, 4, 5, but achieves slightly better response time due to the improved ranking of requests.

### Burst Workload

For the Burst workload, we do not achieve any difference between SPRPT, LSPRPT, and our DTRPRT policies, as the information given by having more policies at decision time overwhelms the benefits of having information about memory usage. The policies still outperform FCFS and SJF, highlighting the value of preemptions.

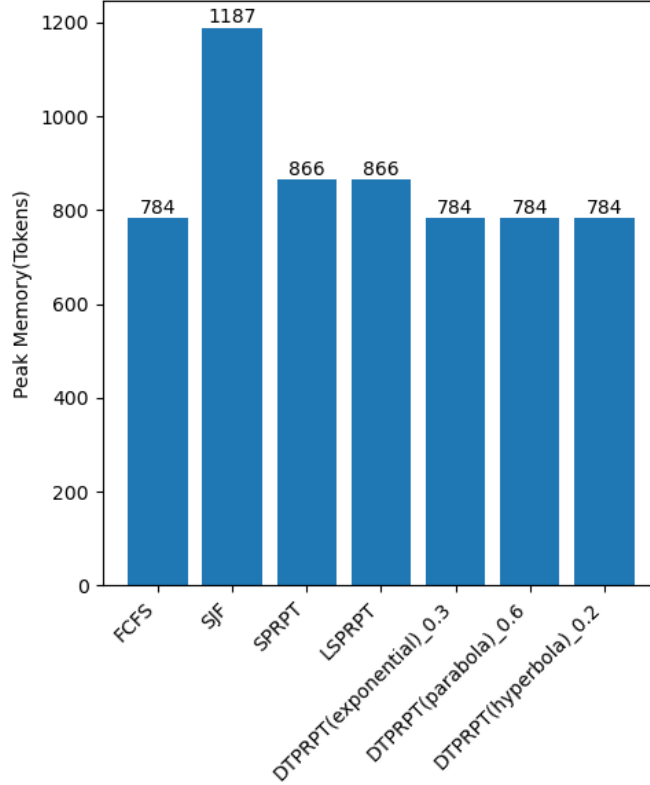


Figure 5: Peak Memory usage for realistic-normal workload with  $\sigma = 0.5$

## 6 Future Directions

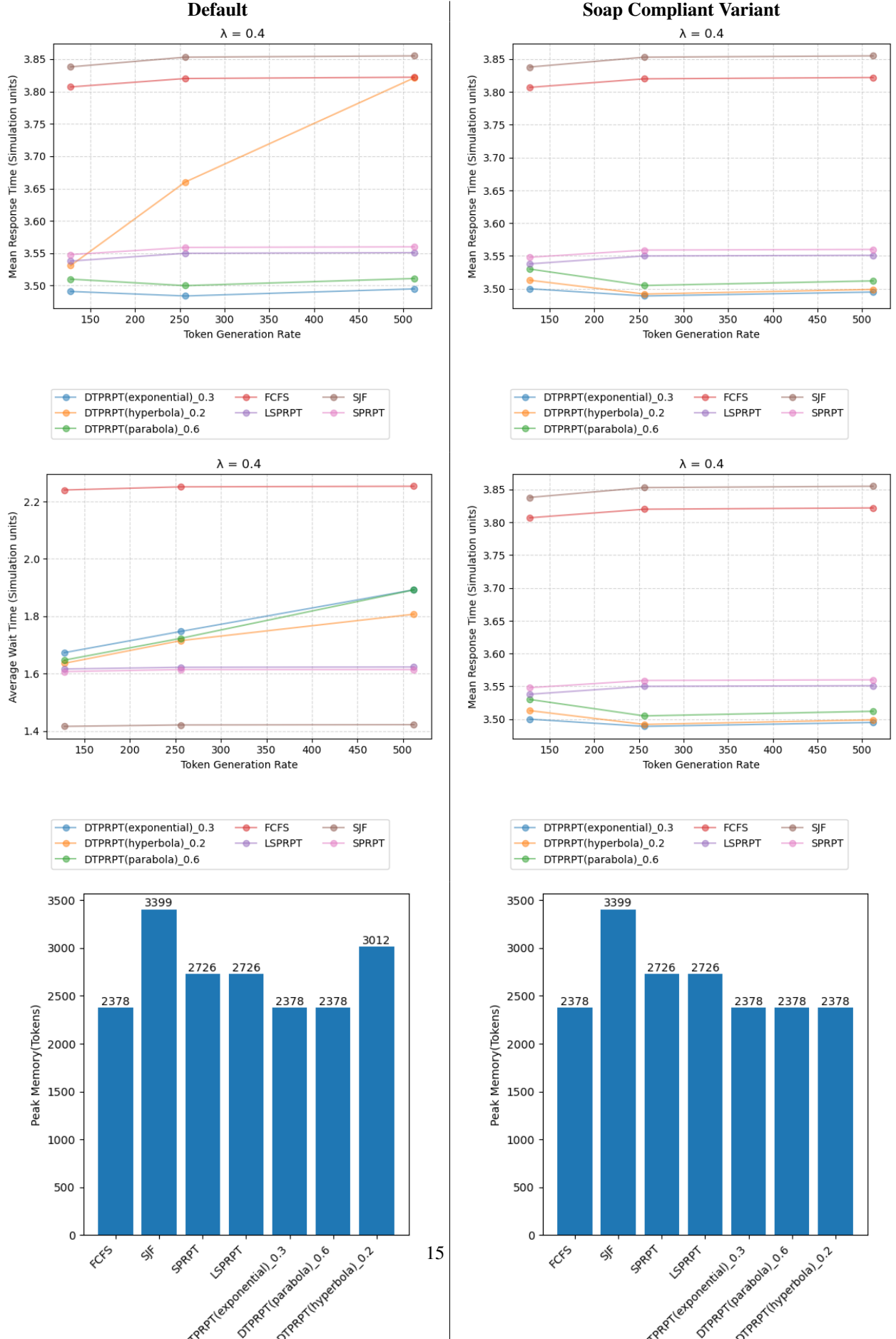
While the initial choice of our functions is inspired by memory growth inductive biases, tuning the exact parameters to the workload becomes cumbersome. An adaptive threshold that tunes the parameters used over time is a promising extension of MEM-PREM, especially since statistics are easily accessible within the first few time steps of a slow-changing workload.

Our experiments have been limited to only one server to be consistent with our theoretical setup. The area of multi-server systems is nascent in the theoretical sense, and such developments especially for SOAP-like framework can help bridge the gap between theory and experimental work.

Lastly, memory growth was assumed to grow linearly with the number of tokens, however, in practice, the prefilling stage of the key-value cache complicates such a model. More work to model this mathematically can be used to better inform our policies when used on real LLM inference servers.

## 7 Conclusion

We have inspired a new line of work in the direction of memory-informed policies beyond static thresholds defined by hyperparameter tuning. We firstly do this with dynamic threshold and find two of our initial hypothesis functions:

Table 2: Dynamic Threshold Experiments for  $\lambda = 0.4$  under exponential-exponential workload

parabolic and exponential consistently outperform or equal the performance of the previous state-of-the-art across a diverse range of workloads. We subsequently consider even smoother integrations of memory into the policy decision making via parametric penalty functions. Both our exponentiated and sigmoid implementations use simple memory heuristics. We back up the class of MEM-PREM policies by extending the closed formula for mean response time for both dynamic threshold and penalty functions. This in turn opens up the design-space for new memory-based policies that can optimally navigate the tradeoff space of memory usage and response time.

## References

- [1] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. Don’t stop me now: Embedding based scheduling for llms, 2024.
- [2] Ziv Scully and Mor Harchol-Balter. Soap bubbles: Robust scheduling under adversarial noise. In *2018 56th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 144–154, 2018.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [4] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline, 2023.
- [5] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. Soap: One clean analysis of all age-based scheduling policies, 2018.
- [6] J. Baxter. A model of inductive bias learning. *Journal of Artificial Intelligence Research*, 12:149–198, March 2000.
- [7] Michael Mitzenmacher. Scheduling with predictions and the price of misprediction, 2019.
- [8] Aaron Grattafiori et al. The llama 3 herd of models, 2024.
- [9] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. [https://github.com/tatsu-lab/stanford\\_alpaca](https://github.com/tatsu-lab/stanford_alpaca), 2023.

## 8 Appendix

### 8.1 Tables

Below are our results for a realistic-normal workload with arrival rate 0.6 and token generation rate 256.

| Algorithm                 | Completed Requests | Avg. Wait Time (s) | Mean Resp. Time (s) | Peak Mem. (MB) | Preemptions |
|---------------------------|--------------------|--------------------|---------------------|----------------|-------------|
| FCFS                      | 149732             | 1.278              | 2.285               | 784            | 0           |
| SPRPT                     | 149732             | 1.264              | 2.285               | 866            | 1478        |
| LSPRPT                    | 149732             | 1.264              | 2.285               | 866            | 1400        |
| SJF                       | 149732             | 1.256              | 2.801               | 1137           | 37465       |
| DTPRPT (hyperbola, 0.2)   | 149732             | 1.264              | 2.284               | 885            | 1284        |
| DTPRPT (parabola, 0.6)    | 149732             | 1.264              | 2.285               | 784            | 1445        |
| DTPRPT (exponential, 0.3) | 149732             | 1.265              | 2.282               | 784            | 1046        |
| PSPRPT (exponentiated)    | 149732             | 1.270              | 2.277               | 784            | 0           |
| PSPRPT (sigmoid)          | 149732             | 1.266              | 2.273               | 784            | 0           |

Table 3: Performance comparison across scheduling algorithms for error 0.3 under Realistic-Normal workload

| Algorithm                 | Completed Requests | Avg. Wait Time (s) | Mean Resp. Time (s) | Peak Mem. (MB) | Preemptions |
|---------------------------|--------------------|--------------------|---------------------|----------------|-------------|
| FCFS                      | 149732             | 1.278              | 2.285               | 784            | 0           |
| SPRPT                     | 149732             | 1.265              | 2.343               | 866            | 6945        |
| LSPRPT                    | 149732             | 1.265              | 2.338               | 866            | 6232        |
| SJF                       | 149732             | 1.260              | 2.805               | 1187           | 37415       |
| DTPRPT (hyperbola, 0.2)   | 149732             | 1.265              | 2.336               | 989            | 6024        |
| DTPRPT (parabola, 0.6)    | 149732             | 1.265              | 2.342               | 784            | 6873        |
| DTPRPT (exponential, 0.3) | 149732             | 1.266              | 2.332               | 784            | 5670        |
| PSPRPT 1 (exponentiated)  | 149732             | 1.270              | 2.277               | 784            | 0           |
| PSPRPT 2 (sigmoid)        | 149732             | 1.269              | 2.276               | 784            | 0           |

Table 4: Performance comparison across scheduling algorithms for 0.5 under Realistic-Normal workload

| Algorithm                 | Completed Requests | Avg. Wait Time (s) | Mean Resp. Time (s) | Peak Mem. (MB) | Preemptions |
|---------------------------|--------------------|--------------------|---------------------|----------------|-------------|
| FCFS                      | 149732             | 1.278              | 2.285               | 784            | 0           |
| SPRPT                     | 149732             | 1.266              | 2.406               | 926            | 11944       |
| LSPRPT                    | 149732             | 1.266              | 2.396               | 926            | 10683       |
| SJF                       | 149732             | 1.263              | 2.806               | 1137           | 37044       |
| DTPRPT (hyperbola, 0.2)   | 149732             | 1.267              | 2.394               | 1053           | 10399       |
| DTPRPT (parabola, 0.6)    | 149732             | 1.266              | 2.404               | 926            | 11819       |
| DTPRPT (exponential, 0.3) | 149732             | 1.267              | 2.390               | 926            | 10045       |
| PSPRPT 1(exponentiated)   | 149732             | 1.270              | 2.277               | 784            | 0           |
| PSPRPT 2 (sigmoid)        | 149732             | 1.270              | 2.277               | 784            | 0           |

Table 5: Performance comparison across scheduling algorithms for error 0.7 under Realistic-Normal workload

### 8.2 Hyperparameter Tuning

For the Dynamic Threshold Policy, we do some initial hyperparameter tuning to see which are the best values of  $h$  for each of our hypothesis functions, by doing a simulation of 10000 units of time.

The figures can be found in Figures 6, 7, 8. While there is no clear optimum, we see that a middle value of roughly  $h = 0.6$  suits the design of the parabola, while smaller values of  $h$  give faster memory response time for Hyperbolic and Exponential of  $h = 0.2$ , and  $h = 0.3$  respectively.

## 9 Figures

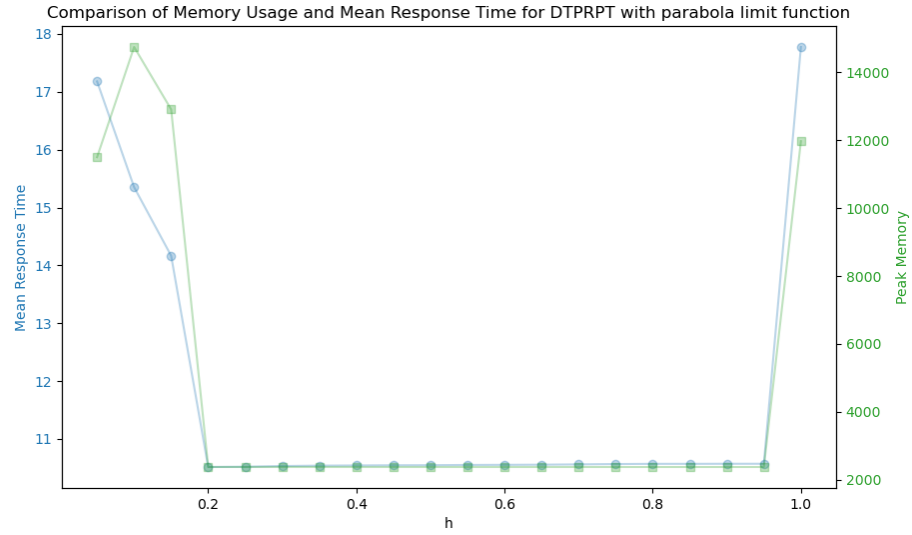


Figure 6: Initial Hyperparameter Tuning of Parabolic Dynamic Threshold

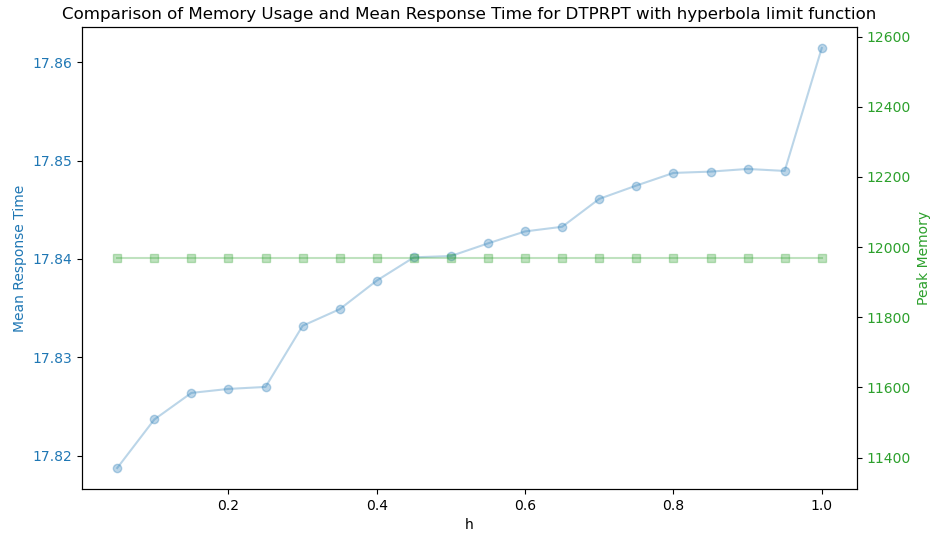


Figure 7: Initial Hyperparameter Tuning of Hyperbolic Dynamic Threshold

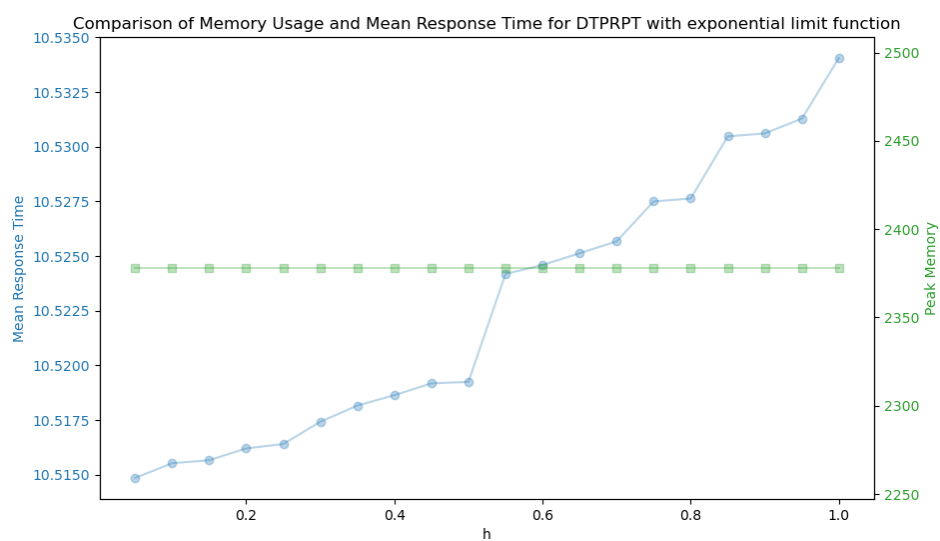


Figure 8: Initial Hyperparameter Tuning of Exponential Dynamic Threshold